

第 1 章：数据结构与算法绪论

1.1 数据结构基本概念

数据结构是相互之间存在一种或多种特定关系的数据元素的集合,包括逻辑结构与物理结构。逻辑结构分为集合、线性结构、树形结构和图形结构。

1.2 复杂度分析原理

大 O 表示法用于评估算法随着输入规模增长时,时间与空间资源的消耗趋势与渐进上界。分析时通常考虑最坏时间复杂度,以保证程序的性能稳定性。

第 2 章：线性表

2.1 顺序表的内存分布

顺序表利用一段物理地址连续的存储单元依次存储数据，支持 $O(1)$ 的随机访问，但在插入和删除时需要移动大量元素，操作效率较低。

2.2 链表的指针机制

链表通过节点中的指针链接非连续的内存空间，实现了 $O(1)$ 的插入与删除，但由于其物理地址不连续，无法支持随机访问，只能顺序遍历。

第 3 章：栈与队列

3.1 栈的后进先出 (LIFO)

栈是一种只允许在表尾进行插入或删除操作的线性表，常用于函数调用、表达式求值与递归算法中保存上下文。

3.2 队列的先进先出 (FIFO)

队列允许在表的一端进行插入，在另一端进行删除，广泛应用于广度优先搜索、操作系统任务调度及打印机缓冲队列。

第 4 章：串与数组

4.1 字符串匹配 (KMP 算法)

KMP 算法通过构建 next 数组，在字符串匹配失败时避免主串指针回溯，将时间复杂度优化至 $O(n+m)$ ，是文本处理领域的核心算法。

4.2 多维数组与矩阵压缩

特殊矩阵（如对称矩阵、稀疏矩阵）可以通过特定的一维数组映射公式进行压缩存储，从而

第 5 章：树与二叉树

5.1 二叉树的性质与遍历

二叉树的每个节点最多有两个子树。深度优先遍历包括前序、中序和后序，广度优先使用层序遍历。它们是处理层次数据的基础。

5.2 哈夫曼树与数据编码

哈夫曼树是带权路径长度最短的二叉树，其产生的哈夫曼编码是一种前缀编码，常用于数据无损压缩，极大地提高了存储效率。

第 6 章：图论基础

6.1 图的存储结构

图的存储主要分为邻接矩阵（适合稠密图）和邻接表（适合稀疏图），用于表示顶点关系与边权重，是复杂网络模型的核心表现方式。

6.2 DFS 与 BFS 遍历

深度优先搜索(DFS)基于栈实现回溯，广度优先搜索(BFS)基于队列实现层级推进，两者是解决连通性、路径搜索等图算法的基础。

第 7 章：最短路径与最小生成树

7.1 Dijkstra 最短路径算法

Dijkstra 算法基于贪心策略，通过不断松弛边，求解非负权图中单源最短路径问题，广泛应用于地图导航系统。

7.2 Kruskal 最小生成树算法

Kruskal 算法通过并查集检测环路，按权值从小到大选取边，以最小代价连通图中所有顶点，用于网络设计与成本优化。

第 8 章：查找算法

8.1 二分查找的核心逻辑

二分查找要求线性表必须采用顺序存储且按关键字有序，每次比较使搜索范围减半，时间复杂度为 $O(\log n)$ ，效率极高。

8.2 哈希表与冲突解决

哈希表通过散列函数映射地址，解决冲突的常用方法包括开放定址法与链地址法，实现了平均 $O(1)$ 的查找速度。

第 9 章：排序算法

9.1 基础排序 (冒泡与插入)

基础排序算法逻辑简单但效率较低，平均时间复杂度通常为 $O(n^2)$ ，仅适合小规模数据处理或作为高级排序算法的子过程。

9.2 高级排序 (快排与归并)

快速排序基于分治法与基准元划分；归并排序基于合并有序子序列。两者在处理大规模数据时具有显著的时间性能优势。

第 10 章：高级数据结构

10.1 B 树与 B+树

B 树是多路平衡查找树；B+树将所有数据存储在叶子节点并形成链表，是数据库索引、操作系统文件系统底层核心结构。

10.2 红黑树自平衡原理

红黑树通过颜色约束和旋转操作（左旋/右旋）保持近似平衡，保障了在最坏情况下查询、插入、删除操作的对数级性能。